

検知機能



Thought for 2m 59s >

了解。ここはちゃんと設計しておかないと、V2 全体の使い勝手が一気に落ちます。

①「自動検知」以外に良い方法は？

結論から言うと：

- ベストは「自動候補 + ユーザ確認（オートコンプリート）」方式
- 「完全自動で黙って決め打ち」は **やめた方がいい** です

理由：

1. **CSV の設計がバラバラ**
 - `treatment`, `arm`, `variant`, `plan`, `pattern` など名前がカオス
 - `outcome` も `y`, `revenue`, `ltv`, `delta_yen`, `converted` など
→ 100%当てるのは不可能。外した時の事故コストが高い。
2. **ガバナンス系は間違えると危険**
 - Fairness / Compliance で列を取り違えると
「差別しているように見えるレポート」を誤出力しかねない
→ 自動候補を **UI** 上で必ず明示して、ユーザに **OK** を押させるべき
3. **世界レベルの実装もほぼ同じ**
 - Meta / Google も「Auto detected: X」「Change column」みたいに *suggestion* として出して、最終決定は人がする。

なので設計は：

Rule-based auto detection (候補生成) → UI で選択済みにセット → ユーザが確認・修正
を前提にした方がいいです。

② 自動検知（候補提示）でいく前提の仕様（Claude 向け）

そのまま貼れるように書きます。

やることは大きく 2 つです。

1. バックエンド：列ロール推定 **API** (`treatment` / `outcome` / `id` / `sensitive` など)
2. フロントエンド：ボタン 1 つでフォームを埋める **UI**

Claude への依頼テキスト

0. ゴール

CQOx の V2 (Governance Center / Experiment Studio) に

「アップロード済みデータから列ロールを自動推定して、フォームにプリセットする機能」を追加したい。

- 完全自動ではなく「自動候補+ユーザ確認」方式
- 既存の `public API` は壊さない。追加エンドポイントで対応
- 変更範囲は **最小限 (diff が追えるレベル)** にすること

1. バックエンド変更 (FastAPI, Python)

1-1. 新規モジュール追加

- 追加ファイル (例名。プロジェクト構成に合わせて調整してよい)
 - `backend/engine/column_inference.py`

```
# ファイル作成
mkdir -p backend/engine
touch backend/engine/column_inference.py
```

1-2. 列ロール推定ロジック

column_inference.py に、以下の関数を実装してください。

```
# backend/engine/column_inference.py
from __future__ import annotations
from dataclasses import dataclass
from typing import Dict, List, Optional
import re
import pandas as pd

@dataclass
class ColumnSuggestion:
    role: str          # "treatment" | "outcome" | "id" | "feature" | "sensitive"
    column: str
    score: float        # 0~1 の信頼度
    reason: str         # 簡単な説明 (name match / cardinality etc.)

KEYWORDS = {
    "treatment": ["treatment", "treat", "arm", "variant", "group", "segment", "policy", "campaign"],
    "outcome": ["delta_yen", "revenue", "sales", "ltv", "profit", "y", "conversion", "converted"],
    "id": ["user_id", "customer_id", "member_id", "account_id", "id"],
    "sensitive": ["gender", "sex", "age", "age_group", "prefecture", "region"],
}

def infer_column_roles(df: pd.DataFrame) -> Dict[str, List[ColumnSuggestion]]:
    """
    与えられた DataFrame から、各ロールの列候補を推定する。
    - name によるマッチ
    - unique によるカテゴリ/連続の判定
    - binary / low-cardinality を treatment / sensitive 候補にする
    """
    suggestions: Dict[str, List[ColumnSuggestion]] = {
        "treatment": [],
        "outcome": [],
        "id": [],
        "sensitive": [],
    }

    nunique = df.nunique(dropna=True)

    for col in df.columns:
        lname = col.lower()
        nuniq = int(nunique[col])

        # ID 候補
        if nuniq == len(df) and any(k in lname for k in KEYWORDS["id"]):
            suggestions["id"].append(
                ColumnSuggestion(
                    role="id",
                    column=col,
                    score=0.9,
                    reason=f"unique values ({nuniq}) and name matches ID pattern",
                )
            )

        # treatment 候補 (バイナリ / 低カーディナリティ)
        if nuniq <= 10 and any(k in lname for k in KEYWORDS["treatment"]):
            score = 0.8 if nuniq <= 3 else 0.6
            suggestions["treatment"].append(
                ColumnSuggestion(
                    role="treatment",
                    column=col,
                    score=score,
                    reason=f"{nuniq} unique values and name matches treatment keywords",
                )
            )
```

```

# outcome 候補 (連続 or バイナリ)
if any(k in lname for k in KEYWORDS["outcome"]):
    suggestions["outcome"].append(
        ColumnSuggestion(
            role="outcome",
            column=col,
            score=0.8,
            reason="name matches outcome keywords",
        )
    )

# sensitive 候補 (低カーディナリティ + 名前)
if nuniq <= 20 and any(k in lname for k in KEYWORDS["sensitive"]):
    suggestions["sensitive"].append(
        ColumnSuggestion(
            role="sensitive",
            column=col,
            score=0.7,
            reason=f"{nuniq} unique values and name matches sensitive keywords",
        )
    )

return suggestions

```

- ここでは単純なルールベースで良い。あとでスコアリングを調整できるよう、`score` と `reason` を必ず返すこと。

1-3. FastAPI エンドポイント

既存の dataset 読み込みロジックを再利用して、以下のような API を追加してください。

- 例: `backend/gateway/app.py` にルータを追加

```

# backend/gateway/app.py (既存の FastAPI アプリに追記)
from fastapi import APIRouter, Depends, HTTPException
from pydantic import BaseModel

from engine.column_inference import infer_column_roles
from engine.dataset_store import load_dataset # 既存の読み込み関数を想定

router = APIRouter(prefix="/api/v2")

class ColumnSuggestionDTO(BaseModel):
    role: str
    column: str
    score: float
    reason: str

class ColumnInferenceResponse(BaseModel):
    dataset_id: str
    suggestions: dict[str, list[ColumnSuggestionDTO]]

@router.get("/datasets/{dataset_id}/column-suggestions",
            response_model=ColumnInferenceResponse)
async def get_column_suggestions(dataset_id: str) -> ColumnInferenceResponse:
    """
    アップロード済み dataset から列ルール候補を返す。
    - 404: dataset が存在しない場合
    - 200: 推定結果 (候補がなければ空配列)
    """
    try:
        df = load_dataset(dataset_id) # pandas.DataFrame を返す想定
    except FileNotFoundError:
        raise HTTPException(status_code=404, detail="Dataset not found")

    suggestions = infer_column_roles(df)
    # dataclass -> dict に変換
    payload = {

```

```

    role: [
        ColumnSuggestionDTO(
            role=s.role,
            column=s.column,
            score=s.score,
            reason=s.reason,
        )
        for s in lst
    ]
    for role, lst in suggestions.items()
}
return ColumnInferenceResponse(dataset_id=dataset_id, suggestions=payload)

```

- 既存の `FastAPI(app)` に `app.include_router(router)` を追加すること。

バックエンド起動 (例)

```

cd backend
uvicorn gateway.app:app --reload

```

単体テストの雛形

```

mkdir -p backend/tests
touch backend/tests/test_column_inference.py

```

backend/tests/test_column_inference.py

```

import pandas as pd
from engine.column_inference import infer_column_roles

def test_infer_simple_case():
    df = pd.DataFrame(
        {
            "user_id": [1, 2, 3],
            "treatment": [0, 1, 1],
            "delta_yen": [100, 200, -50],
            "gender": ["male", "female", "male"],
        }
    )
    s = infer_column_roles(df)
    assert any(c.column == "treatment" for c in s["treatment"])
    assert any(c.column == "delta_yen" for c in s["outcome"])
    assert any(c.column == "gender" for c in s["sensitive"])

```

テスト実行

```

cd backend
pytest -q backend/tests/test_column_inference.py

```

2. フロントエンド変更 (React / TypeScript)

対象ページ:

- `Decision Console` はすでに V2 で使っている `dataset` 情報がある想定
- 今回は特に **Governance Center** と **Experiment Studio** で
 - `treatment_column`, `outcome_column`, `id_column`
 - Governance 用の `Sensitive Attributes` を自動プリセットできるようにする。

ここでは例としてファイル名を仮定する:

- `frontend/src/pages/governance/GovernanceCenter.tsx`
- `frontend/src/pages/experiments/ExperimentStudio.tsx`

2-1. API クライアントの追加

// frontend/src/lib/api.ts (既存クライアントに追記)

```

export type ColumnSuggestion = {
    role: string;
    column: string;
}

```

```

    score: number;
    reason: string;
  };

export type ColumnInferenceResponse = {
  dataset_id: string;
  suggestions: Record<string, ColumnSuggestion[]>;
};

export async function fetchColumnSuggestions(
  datasetId: string,
): Promise<ColumnInferenceResponse> {
  const res = await fetch(`/api/v2/datasets/${datasetId}/column-suggestions`);
  if (!res.ok) {
    throw new Error(`Failed to fetch column suggestions: ${res.status}`);
  }
  return res.json();
}

```

2-2. Governance Center の UI 変更

- ページ上部に「対象 Dataset の選択」ドロップダウンを追加
 - 既存の dataset 一覧 API を利用 (`/api/v1/datasets` 等)
- 「列を自動検知」ボタンを追加し、クリック時に:
 - `fetchColumnSuggestions(selectedDatasetId)` を呼ぶ
 - `sensitive` ロールの上位候補を **Sensitive Attributes** のマルチセレクトにプリセット
 - `id` ロール候補を ID 欄にプリセット (必要なら)
- JSON テキストエリアは **Developer mode** として残すが、デフォルトは隠す
 - 一般ユーザは dropdown / multi-select だけで完結するようにする

擬似コード (抜粋) :

```

// GovernanceCenter.tsx (概略)
const [datasetId, setDatasetId] = useState<string | null>(null);
const [autoLoading, setAutoLoading] = useState(false);
const [sensitiveColumns, setSensitiveColumns] = useState<string[]>([]);

const handleAutoDetect = async () => {
  if (!datasetId) return;
  setAutoLoading(true);
  try {
    const res = await fetchColumnSuggestions(datasetId);
    const sensitive = res.suggestions["sensitive"] ?? [];
    // スコア順にソートして上位 N 件を採用
    const top = sensitive.sort((a, b) => b.score - a.score).slice(0, 3);
    setSensitiveColumns(top.map((c) => c.column));
  } finally {
    setAutoLoading(false);
  }
};

// JSX のどこかに
<Button onClick={handleAutoDetect} disabled={!datasetId || autoLoading}>
  列を自動検知
</Button>

<MultiSelect
  label="Sensitive Attributes"
  value={sensitiveColumns}
  onChange={setSensitiveColumns}
  options={allColumnsFromDataset} // dataset schema から
/>

```

2-3. Experiment Studio の UI 変更

同様に、Experiment Studio の

- Treatment Column

- `Outcome Column`

フィールドに対して、「Auto detect」ボタンを追加し、

```
const handleAutoDetect = async () => {
  if (!datasetId) return;
  const res = await fetchColumnSuggestions(datasetId);
  const treatments = res.suggestions["treatment"] ?? [];
  const outcomes = res.suggestions["outcome"] ?? [];

  if (treatments.length) {
    setTreatmentColumn(treatments[0].column);
  }
  if (outcomes.length) {
    setOutcomeColumn(outcomes[0].column);
  }
};
```

といった形で、最上位候補をフォームに自動セットする。

ユーザはそのまま編集可能。

```
# フロントエンド起動 (例)
cd frontend
npm install
npm run dev
```

3. 動作確認手順

1. 任意の CSV をアップロードし、`dataset_id` を確認
2. `curl` でバックエンドの推定 API を叩く

```
curl http://localhost:8000/api/v2/datasets/<DATASET_ID>/column-suggestions | jq .
```

- `treatment`, `outcome`, `sensitive` にそれぞれの列が入っているか確認
3. フロントエンドで
 - Governance Center で対象 Dataset を選び、「列を自動検知」を押す
→ Sensitive Attributes が自動で埋まること
 - Experiment Studio で「Auto detect」を押す
→ Treatment / Outcome が自動でセットされること

4. 非ゴール / 注意点

- ML モデルを使った高度な列推定は scope 外 (今回はルールベースで十分)
- 既存の因果推論パイプラインや estimator 実装は変更しない
- 推定結果はあくまで候補であり、ユーザが必ず確認できる UI にすること

「What this does」

- **ゴール**
アップロード済み CSV から、Treatment / Outcome / Sensitive などの列候補を自動で推定し、Governance Center と Experiment Studio のフォームを 1 クリックで埋められるようにする。
- **入力**
 - `dataset_id` (既に CQOx に登録されているデータセット ID)
- **出力 / 副作用**
 - `/api/v2/datasets/{dataset_id}/column-suggestions` が、列ロール候補の JSON を返す
 - フロントエンドのボタン操作により、フォームの各フィールドが自動でプリセットされる
 - 既存 API / 推定処理には影響を与えない
- **なぜこのアプローチか**
 - ユーザ負荷の高い「列選択」を自動化しつつ、間違えた時のリスクを避けるために 候補提示 + 確認 にしている

- ルールベースなので、将来の V3 で ML ベースに差し替える時もインターフェースを維持できる
 - 実行 / 検証方法
 - バックエンド: `cd backend && pytest -q backend/tests/test_column_inference.py`
 - ローカル起動後、`curl` で推定結果を確認
 - UI 上で Auto detect ボタンを押し、フォームが期待通りに埋まることを確認する
-

Expert insight (Google/Meta/NASA level)

列ロール推定は「100% 当てる」のではなく「90% のケースを 1 クリックで終わらせる」設計にする。

- 本当に危険なのは「外した時に誰も気づかない」状態であり、それを防ぐには 自動候補の横に理由 (**reason**) を表示し、ユーザに自分の頭でチェックさせる UI が最もコスパが良い。
- Meta や Google の内部ツールでも、Auto detect にはほぼ必ず「Detected by: pattern X, cardinality Y」などの説明がついている。